

Quadruped Locomotion with Central Pattern Generators and Deep Reinforcement Learning

Contents

1	Introduction	1
1.1	Code Structure	2
1.1.1	Environment setup	2
1.1.2	CPG modelling	2
1.1.3	RL integration	2
1.2	Report Structure	2
2	Quadruped Modeling	3
2.1	Coordinate/Reference frame	3
2.2	Control	3
3	Part 1: Central Pattern Generators	4
3.1	Assignment	4
3.2	Tips	5
4	Part 2: Deep Reinforcement Learning	5
4.1	Assignment	6
4.2	Tips	7
5	Submission	7
5.1	Tips	8
6	Generative AI Declaration	8
A	CPG Extensions (Bonus)	10
A.1	Parameters	10
A.2	Gaits	10
A.3	Virtual Model Control	11
A.4	Force Feedback in CPG Equations	11

1 Introduction

In this project we will explore two methods for quadruped locomotion. In the first part, we will implement different gaits with Central Pattern Generators (CPG). This will make you familiar with modelling a biologically-inspired locomotion controller. In the second part, we will design controllers using state-of-the-art deep reinforcement learning (DRL) algorithms. Note that the two parts can be completed in either order, but make sure to start early on part 2 to allow time to run multiple DRL experiments. This project is **35%** of the total grade and the report needs to be submitted by **19 Dec 2025 23:59**.

1.1 Code Structure

The code is hosted at <https://gitlab.epfl.ch/pey/lr-miniproject-2.git>. Here we give details in addition to the README.md. The scripts below can be done in parallel and do not need to be done in sequence.

1.1.1 Environment setup

The env folder contains all files needed for simulating the quadruped robot in pybullet. The structure is similar to the leg environment we used in Weeks 1-3.

- `quadruped_gym_env.py`: Contains the Gym interface where you will implement your Markov Decision Process (MDP) design decisions (i.e. state space, action space, reward function). **Modify this file, see [TODO] tags.**
- `quadruped.py`: Implements many helpful functions for accessing various robot states, solving inverse kinematics, and computing leg Jacobians.
- `quadruped_motor.py`: Motor model for quadrupeds.
- `configs_a1.py`: Contains configuration variables relating to initial positions, kinematics, joint limits, etc.
- `a1_description`: Folder containing all the robot model meshes files.

1.1.2 CPG modelling

- `hopf_network.py`: Provides a CPG class skeleton for various gaits. **Modify this file, see [TODO] tags.**
- `run_cpg.py`: Maps CPG in `hopf_network.py` to joint commands to be executed on an instance of the `quadruped_gym_env` class. **Modify this file, see [TODO] tags.**

1.1.3 RL integration

- `run_sb3.py`: Provides an interface to train a RL agent using RL algorithms based on [stable-baselines3](#). You should review the documentation carefully for information on the different algorithms and training hyperparameters. However, feel free to use alternative reinforcement learning libraries such as Ray RLlib or Tianshou for example.
- `load_sb3.py`: Plays the trained policy to evaluate the performance. **Modify this file, see [TODO] tags.**

1.2 Report Structure

Please format your report in the [IEEE double-column conference format](#) (i.e. As for papers for ICRA or IROS). This project is more open-ended than Miniproject 1 but be sure to include all relevant major parts:

- Introduction
- Methodology
- Results
- Discussion
- Conclusion

The assignment sections below will clarify specific questions you should include, but the overall contents should be self-contained and clear to someone who has not read this document. Be sure to cite any ideas, formulas or reward functions etc that you use from the literature. **The target report length is 6-8 pages in double-column format, excluding references and generative AI declaration and supplementary material. For supplementary material, we allow 3 additional pages in case you have extra diagrams to show but the main plots and graphs should be in the main report. We leave it to your creativity on how you want to organise your report!**

2 Quadruped Modeling

In this section we discuss the quadruped modeling used in pybullet.

2.1 Coordinate/Reference frame

The quadruped has 4 legs, each with 3 joints (Hip, Thigh, Calf), for a total of 12 motors. When querying joint states or sending torques to the motors, we thus have an array of length 12. The legs are numbered 0-3 in the order: Front Right (FR, 0), Front Left (FL, 1), Rear Right (RR, 2), Rear Left (RL, 3). The motor order is always Hip, Thigh, Calf for each leg. See the slides for a clearer visualization.

In many cases, we will be interested in the location of the foot in the leg frame. As an example, consider the position of the Front Right (FR) foot at a “neutral” position (i.e. The foot is directly under the hip) when standing at $0.25m$. The foot position in the leg frame is then $(x, y, z) = (0, -\text{hip_length}, -0.25)m$. Please think about whether the other legs will have the same leg frame coordinates (i.e. Why is y for FR negative? What are the coordinates for our simulation?). The functions `ComputeJacobianAndPosition` and `ComputeInverseKinematics` will return/expect this format.

On this note, recall from lecture the relationship between the joint angles $\mathbf{q}$ and foot position $\mathbf{p}$ for leg i :

$$\mathbf{p} = f(\mathbf{q}) \quad (1)$$

where the function $f(\cdot)$ denotes the forward kinematics of a single leg with respect to the leg frame. Differentiating this relationship gives the foot velocity $\mathbf{v}$:

$$\mathbf{v} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} \quad (2)$$

where $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{3 \times 3}$ is the single-leg Jacobian of the foot position with respect to (w.r.t.) the leg frame. The Jacobian can also be used to map a desired force $\mathbf{F}$ at the end effector (foot) to joint torques:

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q})\mathbf{F} \quad (3)$$

These three relationships will be used repeatedly throughout this project.

2.2 Control

In this project you can choose to implement either/both joint PD control and Cartesian PD control. As a reminder from earlier practicals, for position control in joint space for leg i , we can compute the torques with the following formula:

$$\boldsymbol{\tau}_{\text{joint}} = \mathbf{K}_{p,\text{joint}}(\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_{d,\text{joint}}(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) \quad (4)$$

where $\mathbf{q}_d$ are the desired joint angles, $\dot{\mathbf{q}}_d$ are the desired joint velocities, and $\mathbf{q}$ and $\dot{\mathbf{q}}$ are the current joint angles and joint velocities, respectively. $\mathbf{K}_{p,\text{joint}}$ and $\mathbf{K}_{d,\text{joint}}$ are vectors of proportional and derivative gains. Please note the dimensions of each of these vectors.

To improve tracking performance, Cartesian PD controllers can be added (or used as an alternative, for example usually used in MPC [1]). From the desired joint angle ($\mathbf{q}_d$) and joint velocity ($\dot{\mathbf{q}}_d$) trajectories, we can extract desired foot positions ($\mathbf{p}_d$) and foot velocities ($\mathbf{v}_d$) in the leg frame. Given the current foot position ($\mathbf{p}$) and foot velocity ($\mathbf{v}$), the corresponding motor torques to track the desired quantities can be computed with:

$$\boldsymbol{\tau}_{\text{Cartesian}} = \mathbf{J}^T(\mathbf{q}) \left[\mathbf{K}_{p,\text{Cartesian}}(\mathbf{p}_d - \mathbf{p}) + \mathbf{K}_{d,\text{Cartesian}}(\mathbf{v}_d - \mathbf{v}) \right] \quad (5)$$

where $\mathbf{J}(\mathbf{q})$ is the foot Jacobian at joint configuration $\mathbf{q}$, and $\mathbf{K}_{p,\text{Cartesian}}$ and $\mathbf{K}_{d,\text{Cartesian}}$ are diagonal matrices of proportional and derivative gains.

3 Part 1: Central Pattern Generators

In part 1 of this project we will implement our locomotion controllers based on Central Pattern Generators consisting of coupled Cartesian space Hopf oscillators [2-4]. The equations for oscillator i can be written as follows:

$$\dot{r}_i = \alpha(\mu - r_i^2)r_i \quad (6)$$

$$\dot{\theta}_i = \omega_i + \sum_{j=0}^3 r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \quad (7)$$

where r_i is the current amplitude of the oscillator, θ_i is the phase of the oscillator, $\sqrt{\mu}$ is the desired amplitude of the oscillator, α is a positive constant that controls the speed of convergence to the limit cycle, ω_i is the natural frequency of oscillations, w_{ij} is the coupling strength between oscillators i and j , and ϕ_{ij} is the desired phase offset between oscillators i and j .

Here we will split the natural frequency ω_i between ω_{swing} and ω_{stance} , where the phase variable for oscillator i will dictate the stance/swing status of leg i . We will define swing phase (foot in air) as $0 \leq \theta_i \leq \pi$, and stance phase (foot in contact with the ground) as $\pi < \theta_i \leq 2\pi$. Make sure to use the mod operator (%) in python) to keep your phase variables between 0 and 2π .

To evaluate the gait, we define the duty cycle as the time it takes to go through one full cycle of stance + swing phases. The duty ratio D is the duration of one cycle in which the foot is in stance phase T_{stance} over the duration of one full cycle T_{stride} , or $D = T_{stance}/T_{stride}$ [5].

There exist different methods for mapping the CPG states to joint commands. In this assignment we suggest mapping the states to Cartesian foot positions in the leg xz plane. This can be accomplished as follows:

$$x_{foot} = -d_{step}r_i \cos(\theta_i) \quad (8)$$

$$z_{foot} = \begin{cases} -h + g_c \sin(\theta_i) & \text{if } \sin(\theta_i) > 0 \\ -h + g_p \sin(\theta_i) & \text{otherwise} \end{cases} \quad (9)$$

where d_{step} is the step length (Why do we need this?), h is the robot height, g_c is the max ground clearance during swing, and g_p is the max ground penetration during stance.

3.1 Assignment

By tuning the different parameters above, we can produce stable locomotion controllers for the following gaits on flat terrain: **Walk, Trot, Pace, and Bound** (please check the slides for more information). In this assignment, you will need to follow the starter code in `hopf_network.py` to specifically implement/select:

- Each of the coupling matrices ϕ_{gait}
- The swing and stance frequencies ω_{swing} and ω_{stance} (which are gait dependent)
- The mapping from the CPG states to desired quadruped foot positions
- The joint and Cartesian PD controllers to track these desired foot positions

You can test if the controller is working using `run_cpg.py`. Please include the following in your report (we highlight the required plots in blue and videos in red for clarity):

1. A plot of the CPG states ($r, \theta, \dot{r}, \dot{\theta}$) for a trot gait (plots for other gaits are encouraged, but not required). We suggest making subplots for each leg, and make sure these are at a scale where the states are clearly visible (for example 2 gait cycles).
2. A plot comparing the desired foot position vs actual foot position using joint PD with/without Cartesian PD (for one leg is fine). What gains do you use, and how does this affect tracking performance?

3. A plot comparing the desired joint angles vs actual joint angles using joint PD with/without Cartesian PD (for one leg is fine). What gains do you use, and how does this affect tracking performance?
4. Discuss which hyperparameters you found necessary to tune, the highest and lowest resulting body velocity you achieved, the resulting duty cycle/ratio, time duration of one step (time in stance/swing), and resulting Cost of Transport (CoT).
5. Provide two videos of the robot locomoting at different speeds named `GAIT_NAME_HIGH_XXms.mp4` and `GAIT_NAME_LOW_XXms.mp4`, replacing `GAIT_NAME` and `XX` with the average body velocity (after convergence) in *m/s*. At minimum, please show the TROT gait. You can use the `record_video` flag for video recording.
Bonus: We encourage videos/plots/comparisons of other gaits as well.
6. Discuss how the controllers could potentially be extended with different feedback loops and with descending control signals for modulating locomotion (i.e. Change of speeds, gaits, heading, foot placement).
Bonus: The actual implementation is left as a bonus, see Section A.3).

3.2 Tips

- Use the `on_rack` flag to `QuadrupedGymEnv` to freeze the robot base off of the ground. This is useful for visual debugging with low frequency gaits to ensure your coupling is as intended, and that your mapping from the CPG states to joint commands produce reasonable motions. You may also find it helpful to plot/visualize the desired foot trajectories to ensure they are reasonable before proceeding to other parts.
- Adjust/tune/add to functions or parameters step by step rather than adding multiple changes at one go. This allows you to isolate which parts could be the issue for errors.
- So far this framework results in open-loop commands sent to joints (i.e. There is no feedback). For keeping balance, we suggest starting with high frequencies during stance, and 2-4× higher for swing, depending on the gait. Think about what are the slowest/fastest resulting gaits you can produce?

4 Part 2: Deep Reinforcement Learning

In part two of this project you will design a MDP for the quadruped locomotion task, and then train control policies to solve it with state-of-the-art deep reinforcement learning algorithms. Implementing these algorithms is outside the scope of this course, so we provide an interface to two state-of-the-art algorithms, PPO [6] and SAC [7], with a popular reinforcement learning library (`stable-baselines3`) [8]. However, please feel free to do your own research and pick another library.

Here you will start from an OpenAI Gym [9] environment which defines an interface to your MDP for the quadruped robot. Notably, you need to determine what to put in the observation space, action space, and reward function in `quadruped_gym_env.py`. We give you a minimal example with the following and some points to think about:

- Observation space: The observation space `DEFAULT` includes joint angles/velocities and body orientation. Note the upper and lower measurement limits you need to provide for the Gym interface. What other measurements are available on the real robot? What might help the agent choose better actions?
Important: The observation space is normalized before using for RL. Why do you think this is? (Hint: What is the scale of different measurements and how does this affect the ability of the network to learn?)
- Action space: The action space (joint) PD scales the policy network output from `[-1,1]` to joint positions within a smaller subset of the possible range.
Important: Having the policy network output actions in `[-1,1]` is currently standard practice in RL. Why do you think this is? (Hint: What is the scale of different actions we might take in general and what is happening in the neural network?)

- Reward modelling: The reward function FWD_LOCOMOTION currently has several terms to track a desired linear velocity while minimizing energy and other body velocities/orientations. Note that ultimately the reward is a single scalar, so weighting different terms properly is very important to ensure a reasonable value!

Fortunately, there are many papers using deep reinforcement learning-based control for quadruped robots, for example [2, 10–17]. It would be a good idea to check some of these papers (some have already been presented in class!) for inspiration.

4.1 Assignment

Here you will design the observation space, action space, and reward function to train two control policies: **velocity** and **task-specific** with off-the-shelf deep reinforcement learning libraries. For the task specific policy, please choose one of the following tasks (uneven terrain) to focus on: **Gaps, Stairs, Slopes**.

Your report should discuss all of your design choices for the MDP, any modifications to the environment (i.e. Sensor noise, dynamics randomization, adding rough terrain, resets), as well as justification of the learning algorithm hyperparameters you used. We provide hyperparameters for PPO [6] and SAC [7], however these may not be optimal and you should seek to understand the effects of these parameters on the algorithm and resulting policy.

To summarize, include the following main points in your report for discussion of **both policies** (we also added some things to think about for each point; we highlight the required plots in **blue** and videos in **red** for clarity):

- Observation space: What do you include? Why do you think each measurement was important? Did you see performance improvements with adding certain terms? How noisy would these measurements be on hardware? If using CPG-RL, consider placing the CPG states in the observation space. What information do you need to include to move towards a goal?
- Action space: Since we have limited computing resources, we recommend to think carefully about how to structure your action space. Some examples could include: (1) Joint position action space with joint PD control (2) Cartesian space action space with Cartesian PD control (3) Learning to modulate CPG parameters and map these to joint commands (CPG-RL) [2].

We provide an interface for all three of these methods. At least one of these methods should be tested. For the same training conditions (i.e. Observation spaces, reward functions, RL hyperparameters), how do your training curves compare? Qualitatively, how do the resulting control policies look on the robot (i.e. Is the locomotion stable/smooth)?

Bonus: Comparing with one or two other methods gives a bonus.

- Reward function: What is your reward function? What terms do you include? How do you weight the terms to achieve a balanced result?
- Deep reinforcement learning: Which algorithm(s) did you use? Which hyperparameters did you find important? What are some differences between the algorithms, and why/how did this affect the resulting control policy performance? At minimum please explain the provided hyperparameters and neural network architectures.
- Environmental details: What modifications do you make to the environment and why? How did this affect training and policy robustness?
- Robustness: How robust do you think your approach is, and will it transfer successfully to other simulators and the real world (sim-to-sim and sim-to-real)? What are potential difficulties would you encounter with the testing on other simulators and hardware?
- Quantify the performance of your **velocity** controller: Include **plots of relevant robot states, mean reward and mean episode length curves** during the execution of one of your learned policies to go along with **a video** and the **trained weights**.

Please name the video RL_VEL_XXms.mp4 and the trained weights RL_VEL_XXms.pt, replacing XX with the average body velocity (after convergence) in m/s . Similarly at minimum, please show the TROT gait.

Bonus: We encourage videos/plots/comparisons of other gaits as well.

Important: What is the average body velocity? What do you conclude about your reward function? Can you train a policy that you can command at test time to achieve any arbitrary speed (if not, discuss how you would do so)? What is the Cost of Transport? Include the same discussion as in the CPG section relating to resulting duty cycle/ratio, time duration of one step (time in stance/swing) etc.

- Quantify the performance of your **task-specific** controller (i.e. Gaps, Stairs, and/or Slopes): **This is the main objective of this part of the project.** Include **plots of relevant robot states, mean reward and mean episode length curves** during the execution of one of your learned policies to go along with **a video** and the **trained weights**.

Please name the video `RL_TASK.mp4` and the trained weights `RL_TASK.pt`, replacing TASK with the specific chosen task. Similarly at minimum, please show at least one of the tasks.

Bonus: We encourage videos/plots/comparisons of more than one task.

Important: What is the average velocity? What is the maximum gap length and gap density (how close gaps are together) you can achieve? What are the most challenging stairs/slopes the robot can traverse? Discuss what you would do to make the policy work on hardware, and what you think are the maximum limits for the robot.

4.2 Tips

- Focus on setting up a **basic working RL workflow first** before exploring more complex stuff!
- If you have installation issues or training is taking a long time on your local machine, you may want to consider resources such as Google Collab (free access to a GPU).
- During training you can monitor `ep_len_mean` (average episode length) and `ep_rew_mean` (average episode reward). The environment automatically resets after 10 simulated seconds, so a policy that does not cause the agent to fall should average 1000 for `ep_len_mean`. The average episode reward will depend on your reward function.
- You can periodically use `load_sb3.py` to check the training curves for `ep_len_mean` and `ep_rew_mean`, as well as load the most recent intermediate control policy checkpoint being saved at the stated path. Training should converge relatively quickly (i.e. Within 1 million observation timesteps in most ideal cases). If you are training for much longer without improvement you should check that your MDP is reasonable.
- Your resulting control policies may not look optimal. This is ok! Bloopers are encouraged to evaluate not-so-good scenarios. Why do you think the agent has converged to the policy you see, and what could you do to improve it?
- When reporting results, make sure to train several times, varying the random seed, and average the training curves. This is very important to ensure reproducibility and avoid overfitting! Research has shown that this highly affects the results, see [18].
- Researchers currently use many techniques to try to ensure the agent learns a robust controller that will maximize its probability of success on a new environment and generalizability. You may want to change the environment with some of the following possible ideas: Domain randomization (adding/subtracting mass/inertia from various links), varying the coefficient of friction, varying the terrain (add boxes), adding noise to the observation etc.

5 Submission

When submitting, zip your report, your code, and report, together named `lr_mp2_group_{group_number}.zip` with your group number. Your code should include **code files only** (e.g., no `venv` directory for the virtual environment). Videos and weights should be named with the descriptive names discussed in Section 3.1 and 4.1. The structure of your submission folder should look like this:

```

lr_mp2_group_{group number}.zip
|- report.pdf           # Your report in PDF
|- env/                 # Script with your changes
|   |- __init__.py
|   |- configs_a1.py
|   |- hopf_network.py
|   |- quadruped.py
|   |- quadruped_gym_env.py
|   \- quadruped_motor.py
|- utils/                # Script with your changes
|   |- __init__.py
|   |- file_utils.py
|   \- utils.py
|- videos/               # Include relevant videos!
|   |- TROT_HIGH_0.5ms.mp4| # Descriptive name examples
|   |- TROT_LOW_0.5ms.mp4|
|   |- RL_VEL_0.5ms.mp4|
|   |- RL_GAPS.mp4|
|   \- ...               # Extra videos that you may have
|- weights/              # Include relevant weights!
|   |- TROT.zip|         # Descriptive name examples
|   |- RL_VEL_0.5ms.zip|
|   |- RL_GAPS.zip|
|   \- ...               # Extra weights that you may have
|- load_sb3.py           # Script with your changes
|- run_cpg.py            # Script with your changes
|- run_sb3.py            # Script with your changes
\- requirements.txt      # Script with your changes

```

Note that the max submission size is 100 MB, which means you may have to compress your videos.

5.1 Tips

- For writing a good report, it is helpful to assume that the reader has never seen the report before. Ideally, they should be able to understand the flow and logic of the report just by reading it.
- Organize the content clearly with a logical structure, consized explanations and clear visuals to guide the reader.
- Remember to proof-read the report before submission.

6 Generative AI Declaration

The use of generative AI tools for this project is allowed (although optional), given an explicit declaration on its use. Please include a *Generative AI declaration* section in your report if you have used any form of generative AI throughout this project, and answer the following questions. If the use of AI differs heavily between team members, add one declaration per team member.

1. Which aspects of the project did you (attempt to) use generative AI for? This can be anything ranging from code completion, report writing, or even clarifying concepts of the course. Describe your overall strategy of when to use or not to use AI.
2. In which aspects was generative AI the **most** useful? Illustrate with a prompt you used and the resulting answer.

3. In which aspects was generative AI the **least** useful? Illustrate with a prompt you used and the resulting answer.
4. Discuss: the use of generative AI has/has not made this project smoother to complete.
5. Discuss: the use of generative AI has/has not had an impact on my ability to learn and understand the concepts behind this project (can be positive or negative).

References

- [1] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the mit cheetah 3 through convex model-predictive control,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–9.
- [2] G. Bellegarda and A. Ijspeert, “CPG-RL: Learning central pattern generators for quadruped locomotion,” *IEEE Robotics and Automation Letters*, 2022.
- [3] L. Righetti and A. J. Ijspeert, “Pattern generators with sensory feedback for the control of quadruped locomotion,” in *2008 IEEE International Conference on Robotics and Automation*, 2008, pp. 819–824.
- [4] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen, “From swimming to walking with a salamander robot driven by a spinal cord model,” *science*, vol. 315, no. 5817, pp. 1416–1420, 2007.
- [5] H.-W. Park, M. Y. Chuah, and S. Kim, “Quadruped bounding control with variable duty cycle via vertical impulse scaling,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 3245–3252.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.
- [8] A. Raffin, A. Hill, M. Ernestus, A. Gleave, A. Kanervisto, and N. Dormann, “Stable baselines3,” <https://github.com/DLR-RM/stable-baselines3>, 2019.
- [9] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [10] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, 2019.
- [11] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, 2020.
- [12] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, “Sim-to-real: Learning agile locomotion for quadruped robots,” in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018.
- [13] A. Iscen, K. Caluwaerts, J. Tan, T. Zhang, E. Coumans, V. Sindhwani, and V. Vanhoucke, “Policies modulating trajectory generators,” in *Conference on Robot Learning*. PMLR, 2018, pp. 916–926.
- [14] X. B. Peng, E. Coumans, T. Zhang, T.-W. Lee, J. Tan, and S. Levine, “Learning agile robotic locomotion skills by imitating animals,” 2020.
- [15] G. Bellegarda, Y. Chen, Z. Liu, and Q. Nguyen, “Robust high-speed running for quadruped robots via deep reinforcement learning,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 10 364–10 370.

- [16] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 91–100.
- [17] G. Bellegarda, M. Shafiee, and A. Ijspeert, “Allgait: Learning all quadruped gaits and transitions,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.04787>
- [18] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
- [19] M. Ajallooeian, S. Pouya, A. Sproewitz, and A. J. Ijspeert, “Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion,” in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 3321–3328.
- [20] M. Ajallooeian, S. Gay, A. Tuleu, A. Spröwitz, and A. J. Ijspeert, “Modular control of limit cycle locomotion over unperceived rough terrain,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013, pp. 3390–3397.
- [21] D. Owaki, M. Goda, S. Miyazawa, and A. Ishiguro, “A minimal model describing hexapedal interlimb coordination: The tegotae-based approach,” *Frontiers in Neurorobotics*, vol. 11, p. 29, 2017.
- [22] D. Owaki, T. Kano, K. Nagasawa, A. Tero, and A. Ishiguro, “Simple robot suggests physical interlimb communication is essential for quadruped walking,” *Journal of The Royal Society Interface*, vol. 10, no. 78, p. 20120669, 2013.

A CPG Extensions (Bonus)

In this section we list possible extensions to the above required assignment. **Note that no part of this section is required, but we leave it as a bonus if you would like to improve the performance of your controllers.**

A.1 Parameters

Most of the parameters in Equations 6-9 have been pre-tuned to work for all of the gaits, however, we encourage you to explore these further for better gait-specific performance. Such parameters involve the mapping from CPG states to desired foot shape such as:

- `ground_clearance`: Foot swing height
- `ground_penetration`: Foot stance penetration into the ground
- `robot_height`: Nominal robot height
- `des_step_len`: Step length
- `coupling_strength`: Weights of w_{ij} matrix in Equation 7
- α : Convergence to limit cycle
- Joint PD and Cartesian PD gains

Which of these do you change and how does this help with performance?

A.2 Gaits

What are some other gaits you think you could produce with the same framework (i.e. Pronk, gallop)? How about omnidirectional locomotion (i.e. Going backwards, laterally etc.)? What needs to be modified?

A.3 Virtual Model Control

Virtual Model Control (VMC; what you saw in Miniproject 1) and reflexes have been used in [19, 20] to improve performance and robustness at variable speeds, as well as the ability to turn and locomote over uneven terrain. Please check in the above cited papers, where some of the key items to add would be:

- Attitude control (attach springs to body to keep orientation - reject pitch and roll variations)
- Lateral angle of attack control (helpful over rough terrain)
- Direction control (maintain or change body heading)
- Stumbling correction reflex (when hitting an obstacle during swing phase)

You can test the robustness of your implementation by varying the friction coefficient of the ground and adding random boxes of varying height to the environment.

A.4 Force Feedback in CPG Equations

Another suggestion for improving robustness is to add feedback directly into the CPG equations. One biology-inspired idea is to add terms based on the normal forces (“Tegotae”) in the phase equations, with more details in [21, 22].